

MACHINE LEARNING IN BIOINFORMATICS

PRIOR-DATA FITTED NETWORKS

Philipp Benner
philipp.benner@bam.de

VP.1 - eScience
Federal Institute of Materials Research and Testing (BAM)

June 10, 2026

BAYESIAN PREDICTION

PREDICTION AND PARAMETER ESTIMATION

- In supervised learning, our main goal is to predict the response y^* for a new input x^*
- A common strategy is to first estimate model parameters θ from training data $x = (x_i)_{i=1}^n$ and $y = (y_i)_{i=1}^n$
- In the Bayesian framework, this is done via the posterior distribution $\text{pr}(\theta | x, y)$
- One simple point estimate is the *maximum a posteriori (MAP) estimate*

$$\hat{\theta} = \arg \max_{\theta} \text{pr}(\theta | x, y)$$

- Other choices, such as the *posterior expectation*, are possible; MAP estimates are often used because they are computationally simple

- MAP selects the parameter value with the highest posterior probability
- Using Bayes' theorem, we obtain

$$\begin{aligned}\hat{\theta} &= \arg \max_{\theta} \text{pr}(\theta | \mathbf{x}, \mathbf{y}) \\ &= \arg \max_{\theta} \frac{\text{pr}(\mathbf{x}, \mathbf{y} | \theta) \text{pr}(\theta)}{\text{pr}(\mathbf{x}, \mathbf{y})} \\ &= \arg \max_{\theta} \text{pr}(\mathbf{x}, \mathbf{y} | \theta) \text{pr}(\theta)\end{aligned}$$

- Hence, MAP combines the likelihood with the prior; for a flat prior, it reduces to the *maximum likelihood estimate (MLE)*

POSTERIOR PREDICTIVE DISTRIBUTION

- Instead of selecting a single parameter value, we can average predictions over the posterior
- For a new input x^* , this yields the posterior predictive distribution

$$\begin{aligned}\text{pr}(y^* | x^*, x, y) &= \int \text{pr}(y^* | x^*, \theta, x, y) \text{pr}(\theta | x, y) d\theta \\ &= \int \text{pr}(y^* | x^*, \theta) \text{pr}(\theta | x, y) d\theta\end{aligned}$$

- Hence, predictions are weighted by how plausible each parameter value is under the posterior
- This is a Bayesian alternative to plug-in prediction with a point estimate such as MAP

PREDICTIVE APPROXIMATION

- The posterior predictive integral is often intractable

$$\text{pr}(\mathbf{y}^* | \mathbf{x}^*, \mathbf{x}, \mathbf{y}) = \int \text{pr}(\mathbf{y}^* | \mathbf{x}^*, \theta) \text{pr}(\theta | \mathbf{x}, \mathbf{y}) d\theta$$

for example when the posterior has no closed form or θ is high-dimensional

- Common strategies first approximate or sample from $\text{pr}(\theta | \mathbf{x}, \mathbf{y})$, e.g. the Laplace approximation, Variational Bayes, or MCMC
- With samples $\theta^{(1)}, \dots, \theta^{(S)}$, we use

$$\text{pr}(\mathbf{y}^* | \mathbf{x}^*, \mathbf{x}, \mathbf{y}) \approx \frac{1}{S} \sum_{s=1}^S \text{pr}(\mathbf{y}^* | \mathbf{x}^*, \theta^{(s)})$$

WHY USE A PFN?

- If the observable joint density were tractable,

$$\text{pr}(y^* | x^*, x, y) = \frac{\text{pr}(x, y, x^*, y^*)}{\int \text{pr}(x, y, x^*, \tilde{y}) d\tilde{y}},$$

so going from the joint to the conditional would be straightforward.

- In practice, Bayesian models are often specified through latent variables

$$\text{pr}(\theta, x, y, x^*, y^*) = \text{pr}(\theta) \text{pr}(x, y, x^*, y^* | \theta),$$

which leads to

$$\text{pr}(y^* | x^*, x, y) = \int \text{pr}(y^* | x^*, \theta) \text{pr}(\theta | x, y) d\theta,$$

and this posterior inference must be solved again for each new dataset.

- PFNs amortize this cost: they are trained offline on sampled tasks and then approximate the posterior predictive in a single forward pass [Müller et al., 2021, Müller et al., 2025].

PFN CAPACITY AND LIMITS

- A PFN does *not* need to copy the parameterization of the generative model.
- The practical requirement is enough expressive power to approximate the posterior predictive family induced by the prior.
- There is no simple one-to-one “complexity matching” rule. Instead, performance depends on inference-network capacity, architecture design, and how well the pre-training tasks match the downstream task family [Cremer et al., 2018, Nagler, 2023].
- Standard PFNs are strongest on small-to-medium datasets; larger context sets increase memory and computational cost, motivating architectural variants for larger datasets [Hollmann et al., 2022, Wang et al., 2025].

PRIOR-DATA FITTED NETWORKS

PRIOR-DATA FITTED NETWORKS (PFN) APPROACH

- Instead of approximating $\text{pr}(\theta | x, y)$ separately for each dataset, PFNs directly learn a conditional predictive model

$$q_{\phi}(y^* | x^*, x, y) \approx \text{pr}(y^* | x^*, x, y)$$

- This mapping is learned offline from the full generative model

$$\text{pr}(\theta, x, y, x^*, y^*) = \text{pr}(\theta) \text{pr}(x, y, x^*, y^* | \theta)$$

- In practice, we first sample $\theta \sim \text{pr}(\theta)$ and then $(x, y, x^*, y^*) \sim \text{pr}(x, y, x^*, y^* | \theta)$
- At test time, the observed examples (x, y) are given as a *context set* and x^* is given as a *query*
- The network uses the context within a single forward pass, i.e. it performs *in-context learning*, and outputs $q_{\phi}(\cdot | x^*, x, y)$ without updating ϕ

- Equivalently, the observable task is drawn from the prior predictive distribution

$$\text{pr}(\mathbf{x}, \mathbf{y}, \mathbf{x}^*, \mathbf{y}^*) = \int \text{pr}(\mathbf{x}, \mathbf{y}, \mathbf{x}^*, \mathbf{y}^* | \theta) \text{pr}(\theta) d\theta$$

- The network receives the context set $(\mathbf{x}, \mathbf{y})$ and the query $\mathbf{x}^*$, and outputs a predictive distribution

$$q_{\phi}(\cdot | \mathbf{x}^*, \mathbf{x}, \mathbf{y})$$

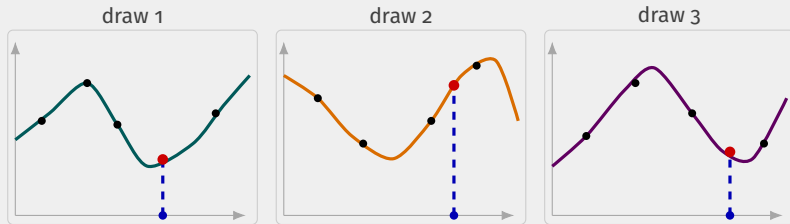
- The sampled target $\mathbf{y}^*$ is not an input to the network; it is used only to evaluate the output distribution in the negative log-likelihood

$$\mathcal{L}(\phi) = \mathbb{E}[-\log q_{\phi}(\mathbf{y}^* | \mathbf{x}^*, \mathbf{x}, \mathbf{y})]$$

- With enough sampled tasks, this trains the network to approximate the posterior predictive distribution

VISUAL EXAMPLE: SAMPLED FUNCTIONS

Example prior: draw latent functions $f^{(s)} \sim \text{pr}(f)$ from a Gaussian process



PFN training uses many such draws: context points (x, y) are black, the query x^* is blue, and the red point y^* provides the training target.

LOSS FUNCTIONS FOR PFNS

- The loss should match the predictive distribution represented by the PFN output

- Regression with point predictions $\hat{y}_\phi(x, y, x^*)$:

$$\mathcal{L}_{\text{MSE}}(\phi) = \mathbb{E} \left[(y^* - \hat{y}_\phi(x, y, x^*))^2 \right]$$

$$\mathcal{L}_{\text{MAE}}(\phi) = \mathbb{E} [|y^* - \hat{y}_\phi(x, y, x^*)|]$$

- Binary classification with binary cross-entropy:

$$\mathcal{L}(\phi) = -\mathbb{E} [y^* \log q_\phi + (1 - y^*) \log(1 - q_\phi)]$$

- Multiclass classification with predictive class probabilities $q_{\phi,j}$:

$$\mathcal{L}(\phi) = -\mathbb{E} \left[\sum_{j=1}^k y_j^* \log q_{\phi,j} \right]$$

- For classification, the output probabilities already quantify uncertainty over classes

REGRESSION LOSSES AS LIKELIHOODS

- MSE corresponds to a Gaussian likelihood with fixed variance

$$q_{\phi}(\cdot | x^*, x, y) = \text{Normal}(\mu_{\phi}, \sigma^2)$$
$$-\log q_{\phi}(y^* | x^*, x, y) = \frac{(y^* - \mu_{\phi})^2}{2\sigma^2} + \text{const}$$

- MAE corresponds to a Laplace likelihood with fixed scale

$$q_{\phi}(\cdot | x^*, x, y) = \text{Laplace}(\mu_{\phi}, b)$$
$$-\log q_{\phi}(y^* | x^*, x, y) = \frac{|y^* - \mu_{\phi}|}{b} + \text{const}$$

- Thus, choosing a regression loss is equivalent to choosing a predictive noise model

REGRESSION: LEARNING UNCERTAINTY

- For the Gaussian process example, the posterior predictive distribution is Gaussian

$$\text{pr}(\cdot | \mathbf{x}^*, \mathbf{x}, \mathbf{y}) = \text{Normal}(\mu^*, \sigma_*^2)$$

- Hence, the PFN can output both a predictive mean and a predictive variance

$$q_\phi(\cdot | \mathbf{x}^*, \mathbf{x}, \mathbf{y}) = \text{Normal}(\mu_\phi(\mathbf{x}, \mathbf{y}, \mathbf{x}^*), \sigma_\phi^2(\mathbf{x}, \mathbf{y}, \mathbf{x}^*))$$

- Training with the negative log-likelihood gives

$$\mathcal{L}(\phi) = \mathbb{E} \left[\frac{1}{2} \log \sigma_\phi^2 + \frac{(\mathbf{y}^* - \mu_\phi)^2}{2\sigma_\phi^2} \right] + \text{const}$$

- The squared error term encourages accurate means, while the $\log \sigma_\phi^2$ term prevents trivially large uncertainties
- If σ_ϕ^2 is fixed, the loss reduces to MSE up to constants

TABPFN PRIOR

- TabPFN [Hollmann et al., 2022] is a *tabular* instance of the PFN idea: it is trained offline to output a predictive distribution

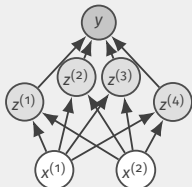
$$q_{\phi}(y^* | x^*, X, y) \approx \text{pr}(y^* | x^*, X, y)$$

where $X \in \mathbb{R}^{n \times p}$ and $x^* \in \mathbb{R}^p$

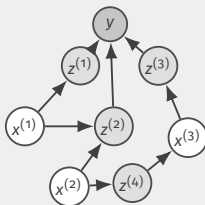
- Unlike a generic PFN, it uses a specific prior over *small tabular classification* tasks, combining SCM-inspired and BNN-like generators with a preference for simple structures
- At test time, the rows of (X, y) act as the context and x^* is the query row; prediction is done by *in-context learning* rather than by fitting a new model
- A single forward pass of the pretrained transformer-like model outputs class probabilities, and performance depends on how well the synthetic prior matches real tabular tasks

TABPFN PRIOR GRAPHS

The TabPFN prior combines two classes of data-generating graphs: BNN-style graphs and SCM-style graphs.



(a) A BNN



(b) An SCM



(c) SCMs sampled from the prior

White nodes are inputs x ; gray nodes are latent variables z ; the darker node is the label y . **SCM allow to easily model complex dependencies between features X_i !**

- **Bayesian Neural Network (BNN):** a neural network whose weights are random variables, for example $W \sim \text{pr}(W)$
- Bayesian prediction averages over posterior weight uncertainty:

$$\text{pr}(y^* | x^*, D) = \int \text{pr}(y^* | x^*, W) \text{pr}(W | D) dW$$

- **Structural Causal Model (SCM):** a directed acyclic graph together with one structural equation per node
- Each variable is generated from its parents and noise:

$$v_j = f_j(\text{pa}_j, \epsilon_j)$$

- In the TabPFN prior, some SCM nodes are observed as features x , while another node is used as the label y

TRANSFORMER ARCHITECTURE

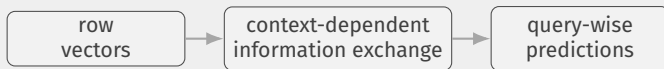
WHAT THE ARCHITECTURE HAS TO DO

For the tabular PFNs considered here, test-time input is an observed context set of rows and a set of query rows. The model returns predictive distributions for the unknown query labels:

$$(D_{\text{ctx}}, X_{\text{qry}}) \mapsto q_{\phi}(Y_{\text{qry}} \mid X_{\text{qry}}, D_{\text{ctx}})$$

with $D_{\text{ctx}} = \{(x_i, y_i)\}_{i=1}^n$ and $X_{\text{qry}} = \{x_j^*\}_{j=1}^m$.

- The number of context examples can vary from task to task
- The order of rows carries no meaning: rows are a set, not a time series
- Different query rows may depend on different context examples
- Inference should be one forward pass, without fitting a new model for the dataset



THE INPUT IS A SET

Because rows form a set, the architecture should respect two symmetries.

- Context order should not matter:

$$q_{\phi}(\cdot | X^*, PX, Py) = q_{\phi}(\cdot | X^*, X, y)$$

for any permutation matrix P acting on the context rows.

- Query order should only reorder the outputs:

$$q_{\phi}(\cdot | PX^*, X, y) = Pq_{\phi}(\cdot | X^*, X, y).$$

- Therefore PFNs for tabular data do not use example-position encodings as language models do.

WHY TURN ROWS INTO TOKENS?

We do not feed raw rows directly into the transformer. Here, a *token* means one fixed-dimensional vector that the transformer layer can update.

Raw tabular input

- Context rows have the form (x_i, y_i)
- Query rows have the form x_j^*
- The target is observed in one case and missing in the other

Vector representation

- Each row is encoded as a vector in $\mathbb{R}^d$
- Context and query rows are mapped into the same vector space
- These row vectors are the transformer *tokens*

For tabular PFNs, one token represents one data row; the row encoder is what turns raw rows into these vectors.

HOW ARE NUMERICAL TOKENS COMPUTED?

For text, a word token is often mapped to a vector by a lookup table. For tabular data, the token is computed by a learned function of the numerical row values.

context row: (x_i, y_i) , query row: x_j^* .

The feature encoder is shared, but label information is added only for context rows.

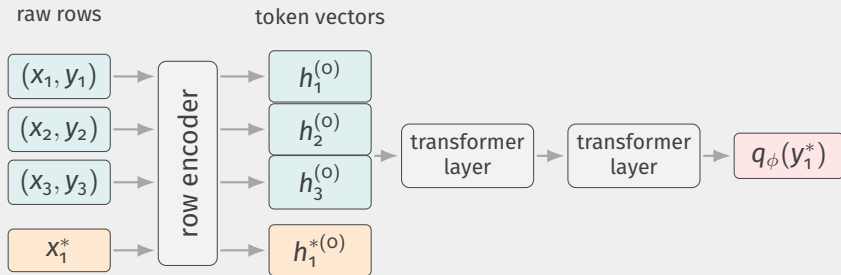
Following the TabPFN implementation, one can write this schematically as

$$h_i^{(o)} = e_\phi^x(x_i) + e_\phi^y(y_i), \quad h_j^{*(o)} = e_\phi^x(x_j^*), \quad h_i^{(o)}, h_j^{*(o)} \in \mathbb{R}^d.$$

The query token is not given a numerical target value; it only receives the feature embedding. All encoder parameters are learned during PFN training.

ENCODING ROWS AS TOKENS

Each raw row is encoded into the initial vector used by the transformer.

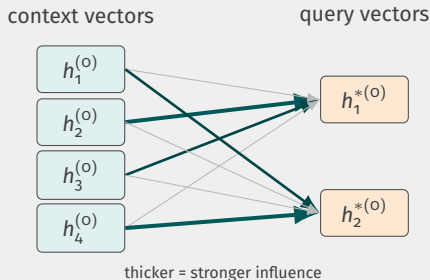


QUERY-SPECIFIC CONTEXT

For a query row x^* , the model should use the context rows that are informative for predicting y^* .

Intuition

- Rows are unordered
- Not every row should matter equally
- Useful rows can change with x^*
- After encoding, each query vector needs its own context summary



ATTENTION AS A SOFT LOOKUP

Attention is the operation used to build such query-specific summaries. For one receiving token h_* , it performs three operations.

1. Compare h^* with each source token h_j and obtain a score s_j
2. Convert scores into weights (softmax)

$$\alpha_j = \frac{\exp(s_j)}{\sum_i \exp(s_i)}, \quad \alpha_j \geq 0, \quad \sum_j \alpha_j = 1$$

3. Return the weighted information from the source tokens

$$o_* = \sum_j \alpha_j v_j$$

Thus, attention is a differentiable lookup: it does not select one row, but reads a weighted mixture of rows.

LEARNED QUERIES, KEYS, AND VALUES

Transformers use a more complex version of attention, where the query, key, and value vectors are learned linear projections of the input tokens.

$$q_* = W_Q h^*, \quad k_j = W_K h_j, \quad v_j = W_V h_j$$

- q_* asks: what is the receiving token looking for?
- k_j describes: when should source token j be read?
- v_j contains: what information source token j provides

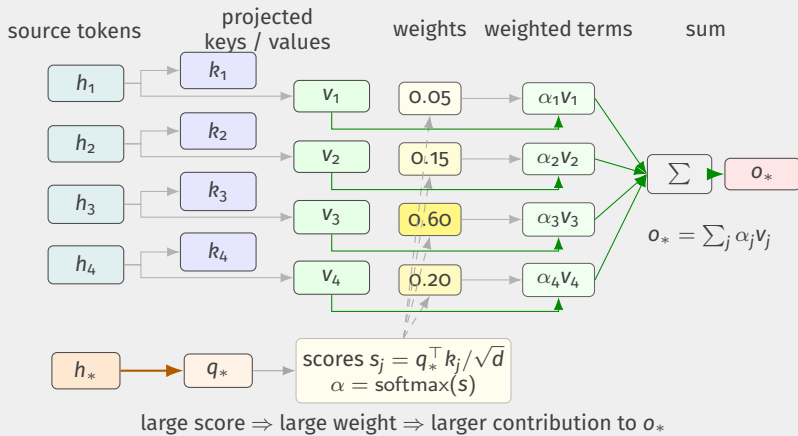
The compatibility score and attention output are

$$s_j = \frac{q_*^\top k_j}{\sqrt{d}}, \quad \alpha_j = \text{softmax}(s)_j, \quad o_* = \sum_j \alpha_j v_j.$$

where d is the token dimension, and the scaling by $\sqrt{d}$ prevents scores from growing too large.

WHAT ONE ATTENTION ROW DOES

For one receiving token $h^{*(\ell)}$, attention builds a weighted summary of the source values.



KERNEL VIEW OF ATTENTION

Attention and kernel regression both compute normalized weighted averages.

Kernel regression

$$\hat{y}(x^*) = \sum_j \alpha_j^{\text{ker}} y_j$$
$$\alpha_j^{\text{ker}} = \frac{K(x^*, x_j)}{\sum_i K(x^*, x_i)}$$

- fixed or hand-chosen similarity K
- values are observed labels y_j

Attention

$$o_* = \sum_j \alpha_j v_j$$
$$\alpha_j = \text{softmax} \left(\frac{q_*^\top k_j}{\sqrt{d}} \right)_j$$

- learned similarity through q_* and k_j
- values v_j are learned representations

This is a structural analogy, not an equivalence: transformer layers learn the representations, the similarity, and the values, and stack this operation many times [Tsai et al., 2019].

MASKED ATTENTION IN PFNS

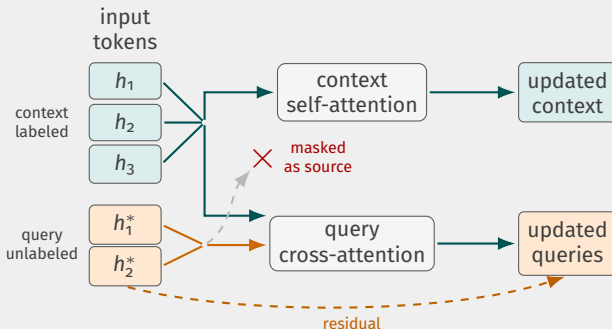
In a PFN, not every token should be allowed to read from every other token.

- Context tokens may exchange information with other context tokens
- Query tokens should read the observed context, because this is the data used for prediction
- Query tokens should not read other query tokens, otherwise the prediction for x_j^* could depend on which other test points are evaluated in the same batch
- The attention mask encodes exactly this information-flow constraint

context $\leftarrow$ context, query $\leftarrow$ context, query $\nleftarrow$ query.

INFORMATION FLOW IN TABPFN

In TabPFN, the attention mask specifies which tokens may be used as sources in one layer [Hollmann et al., 2022].



- Query rows read the context, but not other query rows
- The query's own vector is still preserved by the residual connection

MASKED SELF-ATTENTION

Let $H^{(\ell)} \in \mathbb{R}^{(n+m) \times d}$ be the token matrix in layer ℓ . The mask A implements the information-flow constraint from the previous slide. One attention head uses

$$Q = H^{(\ell)} W_Q, \quad K = H^{(\ell)} W_K, \quad V = H^{(\ell)} W_V$$

and computes

$$\text{Attn}(H^{(\ell)}) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}} + A\right) V$$

where the mask A is

$$A = \begin{pmatrix} \mathbf{0}_{n \times n} & -\infty \cdot \mathbf{1}_{n \times m} \\ \mathbf{0}_{m \times n} & -\infty \cdot \mathbf{1}_{m \times m} \end{pmatrix}.$$

- Each row of the softmax is a distribution over allowed source tokens
- The additive $-\infty$ mask makes forbidden attention weights exactly zero

COMPUTING THE MASK EFFICIENTLY

For the TabPFN mask, only two score blocks are allowed: context reads context, and query reads context. TabPFN evaluates these blocks directly [Hollmann et al., 2022].

Each call uses receiver tokens as queries, and source tokens as keys and values:

$$\begin{aligned} H'_{\text{ctx}} &= \text{MHA}_{\theta}(Q = H_{\text{ctx}}, K = H_{\text{ctx}}, V = H_{\text{ctx}}) && n^2 \text{ scores} \\ H'_{\text{qry}} &= \text{MHA}_{\theta}(Q = H_{\text{qry}}, K = H_{\text{ctx}}, V = H_{\text{ctx}}) && mn \text{ scores} \end{aligned}$$

- Query tokens read context tokens; they do not read other query tokens
- Context tokens do not read query tokens
- The two calls share the same attention parameters, as in TabPFN
- The result matches the masked formulation above, while computing only $n^2 + mn$ instead of $(n + m)^2$ scores

ONE TRANSFORMER BLOCK

Let $\text{MHA}(H^{(\ell)})$ denote the masked multi-head attention output described above.

One transformer block updates the token matrix by two residual sublayers:

$$\begin{aligned}\tilde{H}^{(\ell)} &= \text{LN}\left(H^{(\ell)} + \text{MHA}\left(H^{(\ell)}\right)\right) \\ H^{(\ell+1)} &= \text{LN}\left(\tilde{H}^{(\ell)} + \text{FFN}\left(\tilde{H}^{(\ell)}\right)\right)\end{aligned}$$

The feed-forward network is applied independently to each token:

$$\text{FFN}(h) = W_2 \sigma(W_1 h + b_1) + b_2.$$

- The attention sublayer mixes information across allowed tokens
- The feed-forward sublayer changes each token vector separately
- TabPFN stacks 12 transformer layers; the FFN hidden dimension is 1024 [Hollmann et al., 2022]

CLASSIFICATION HEAD AND PREDICTIVE DISTRIBUTION

After the last transformer layer, each query point has a contextualized representation $h_j^{*(L)} \in \mathbb{R}^d$.

$$z_j = W_{\text{cls}} h_j^{*(L)} + b_{\text{cls}} \in \mathbb{R}^C$$

For a C -class problem, the predictive probabilities are

$$q_\phi(y_j^* = c \mid x_j^*, X, y) = \frac{\exp(z_{jc})}{\sum_{c'=1}^C \exp(z_{jc'})} = \text{softmax}(z_j)_c$$

For all queries at once, this can be written as

$$Z = H_{\text{qry}}^{(L)} W_{\text{cls}}^\top + \mathbf{1} b_{\text{cls}}^\top, \quad P_{\text{cls}} = \text{softmax}(Z)$$

where the softmax is applied row-wise.

- Each row of $P_{\text{cls}} \in [0, 1]^{m \times C}$ is a posterior predictive class distribution
- For regression, the same backbone can be paired with a different output head, e.g. mean and variance parameters

TRAINING LOSS FOR THE TRANSFORMER PFN

For one synthetic dataset

$$D = \{(x_i, y_i)\}_{i=1}^S \sim p(D),$$

we sample a permutation π and split the rows into context and query parts:

$$D_{\text{ctx}} = \{(x_{\pi(i)}, y_{\pi(i)})\}_{i=1}^n$$

$$D_{\text{qry}} = \{(x_{\pi(i)}, y_{\pi(i)})\}_{i=n+1}^S$$

The transformer predicts all query labels in parallel, and the loss for this task is

$$\mathcal{L}(\phi; D, \pi, n) = - \sum_{i=n+1}^S \log q_{\phi}(y_{\pi(i)} | x_{\pi(i)}, D_{\text{ctx}})$$

The full training objective is the expectation

$$\mathbb{E}_{D, \pi, n}[\mathcal{L}(\phi; D, \pi, n)]$$

which matches the Bayesian posterior predictive objective of PFNs [Müller et al., 2021, Hollmann et al., 2022].

PRACTICAL TABPFN DETAILS

- The final TabPFN model in [Hollmann et al., 2022] is a pretrained 12-layer transformer with about 25.82 million parameters
- It is trained once offline on synthetic tasks from the TabPFN prior and then reused for all downstream datasets
- To handle varying feature dimensions $p \leq p_{\max}$, inputs are zero-padded to a fixed size before encoding; TabPFN additionally rescales non-padded features so magnitudes remain comparable across datasets
- At inference time,

$$(X, y, X^*) \longmapsto q_{\phi}(Y^* | X^*, X, y)$$

is computed in one forward pass, with no gradient updates and no hyperparameter optimization

- The paper also reports improved accuracy by ensembling several transformed forward passes, including feature rotations, class-label rotations, and optional power transforms

REFERENCES I

-  BAHDANAU, D., CHO, K., AND BENGIO, Y. (2015).
NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE.
In International Conference on Learning Representations.
-  CREMER, C., LI, X., AND DUVENAUD, D. (2018).
INFERENCE SUBOPTIMALITY IN VARIATIONAL AUTOENCODERS.
In Proceedings of the 35th International Conference on Machine Learning, pages 1078–1086.
-  HOLLMANN, N., MÜLLER, S., EGGENSBERGER, K., AND HUTTER, F. (2022).
TABPFN: A TRANSFORMER THAT SOLVES SMALL TABULAR CLASSIFICATION PROBLEMS IN A SECOND.
arXiv preprint arXiv:2207.01848.

REFERENCES II

-  LEE, J., LEE, Y., KIM, J., KOSIOREK, A., CHOI, S., AND TEH, Y. W. (2019).
SET TRANSFORMER: A FRAMEWORK FOR ATTENTION-BASED PERMUTATION-INVARIANT NEURAL NETWORKS.
In Proceedings of the 36th International Conference on Machine Learning, pages 3744–3753.
-  MÜLLER, S., HOLLMANN, N., ARANGO, S. P., GRABOCKA, J., AND HUTTER, F. (2021).
TRANSFORMERS CAN DO BAYESIAN INFERENCE.
arXiv preprint arXiv:2112.10510.
-  MÜLLER, S., REUTER, A., HOLLMANN, N., RÜGAMER, D., AND HUTTER, F. (2025).
POSITION: THE FUTURE OF BAYESIAN PREDICTION IS PRIOR-FITTED.
In Proceedings of the 42nd International Conference on Machine Learning, pages 81861–81875.

REFERENCES III

-  NAGLER, T. (2023).
STATISTICAL FOUNDATIONS OF PRIOR-DATA FITTED NETWORKS.
In Proceedings of the 40th International Conference on Machine Learning, pages 25660–25676.
-  TSAI, Y.-H. H., BAI, S., YAMADA, M., MORENCY, L.-P., AND SALAKHUTDINOV, R. (2019).
TRANSFORMER DISSECTION: A UNIFIED UNDERSTANDING OF TRANSFORMER'S ATTENTION VIA THE LENS OF KERNEL.
In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pages 4344–4353.
-  VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, Ł., AND POLOSUKHIN, I. (2017).
ATTENTION IS ALL YOU NEED.
In Advances in Neural Information Processing Systems.

REFERENCES IV

-  VIG, J. (2019).
A MULTISCALE VISUALIZATION OF ATTENTION IN THE TRANSFORMER MODEL.
In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, pages 37–42.
-  WANG, Y., JIANG, B., GUO, Y., GAN, Q., WIPF, D., HUANG, X., AND QIU, X. (2025).
PRIOR-FITTED NETWORKS SCALE TO LARGER DATASETS WHEN TREATED AS WEAK LEARNERS.
In Proceedings of The 28th International Conference on Artificial Intelligence and Statistics, pages 1090–1098.